CLOUDFORGE

AWS AMI & EC2 MANAGEMENT PLATFORM

Complete Project Synopsis & Technical Documentation

Project Type	Cloud / DevOps / Web Application
Primary Platform	Amazon Web Services (AWS)
Backend	Node.js + Express + AWS SDK v3
Frontend	HTML / CSS / JavaScript
Deployment	Ubuntu EC2 + Nginx + PM2
AWS Region	ap-south-1 (Mumbai)

Project Vision

CloudForge provides a browser-based control layer for common AWS AMI and EC2 operations. It removes the need to repeatedly navigate the AWS console for routine project operations and demonstrates practical integration of cloud infrastructure, backend APIs, IAM roles, networking, deployment, validation, and DevOps practices.

1. Table of Contents

2. Abstract 3
3. Introduction and Background 3
4. Problem Statement 3
5. Objectives 3
6. Scope of the Project 4
7. Existing System vs Proposed System 4
8. Functional Requirements 4
9. Non-Functional Requirements 5
10. Technology Stack 5
11. System Architecture 5
12. AWS Infrastructure and Networking 5
13. IAM and Security Model 6
14. Backend Design and API Layer 7
15. Frontend Design and Dashboard 8
16. AMI Management Workflow 8
17. EC2 Launch Workflow 9
18. Validation and DryRun Mechanism 11
19. Deployment Architecture 11
20. Complete Step-by-Step Implementation 11
21. Testing and Verification 19
22. Problems Encountered and Resolutions 20
23. Current Project Status 21
24. Limitations 21
25. Future Enhancements 21
26. Project Schedule 22
27. Conclusion 22
28. References 22

2. Abstract

CloudForge is a web-based AWS infrastructure management platform developed to provide a simplified interface for managing Amazon Machine Images (AMIs) and Amazon EC2 instances. The platform is deployed on an Ubuntu-based EC2 server and uses Nginx as a reverse proxy, PM2 for backend process management, Node.js with Express for REST APIs, and the AWS SDK for JavaScript v3 for communication with AWS services.

The system allows a user to retrieve available AMIs, view EC2 instances, configure an EC2 launch request using an AMI, instance type, key pair, security group, and subnet, validate the selected AWS resources, perform an AWS DryRun, launch an instance, and apply CloudForge tags to the newly created resource. The system uses an EC2 IAM role rather than hard-coded AWS access keys, providing a practical demonstration of identity-based access to AWS APIs.

The project combines cloud computing, DevOps, web development, networking, IAM, API development, and infrastructure automation in a single end-to-end application.

3. Introduction and Background

Cloud platforms expose a large number of infrastructure controls through management consoles and APIs. For a student or developer working repeatedly with EC2 instances and AMIs, routine operations can become fragmented across several AWS console screens. CloudForge addresses this workflow by providing one application layer for selected AMI and EC2 operations.

The project was implemented as a real deployed application rather than only a local prototype. The backend runs on an Ubuntu EC2 instance, while Nginx exposes the application over HTTP and forwards API requests to the Node.js service. AWS API requests are authenticated through an IAM role attached to the EC2 server.

4. Problem Statement

Managing EC2 infrastructure manually requires repeated navigation through AMI, instance, key-pair, security-group, subnet, and launch configuration screens. Manual configuration can also result in invalid combinations, such as a subnet and security group belonging to different VPCs. A lightweight project-specific management layer can centralize these operations and validate important parameters before an actual launch.

> **Problem:** Build a reliable web application that can interact with AWS infrastructure programmatically while keeping the user interface simple and the backend responsible for validation and AWS communication.

5. Objectives

- Develop a professional browser-based dashboard for AWS AMI and EC2 operations.
- Retrieve AMIs and EC2 instances dynamically from AWS instead of using static data.
- Provide an EC2 launch workflow based on an existing AMI.
- Load key pairs, security groups, and subnets dynamically from AWS.
- Validate selected AWS resources before attempting a real launch.
- Use AWS DryRun to verify launch permissions/configuration before creating an instance.
- Apply standard CloudForge tags to launched EC2 instances.
- Deploy the complete application on an Ubuntu EC2 server using Nginx and PM2.
- Demonstrate practical IAM, VPC, networking, REST API, and DevOps concepts.

6. Scope of the Project

Current scope: AMI listing/creation workflow, EC2 instance listing, launch configuration, key-pair/security-group/subnet retrieval, validation, DryRun, real EC2 launch, tagging, Nginx reverse proxy, PM2 process management, and IAM-role-based AWS access.

Out of current scope: complete AWS account administration, billing management, multi-region orchestration, full CloudWatch observability, advanced autoscaling management, and production-grade multi-user RBAC.

7. Existing System vs Proposed System

Aspect	Manual / ExistingApproach	CloudForge Approach
AMI discovery	AWS console navigation	AMI data loaded through API
EC2 discovery	AWS console	Dashboard dynamically displays instances
Launch configuration	Multiple AWS screens	Single launch workflow
Validation	Mostly user-driven	Backend validates AMI, key pair, subnet and security group
Permission/config check	Launch directly	DryRun before real launch
Tagging	Manual or omitted	CloudForge applies Name and Project tags
Deployment	Local / manual	Ubuntu EC2 + Nginx + PM2

8. Functional Requirements

ID	Function	Description
FR-01	Health Check	Expose an API endpoint that confirms the backend is running and reports the AWS region.
FR-02	AMI Listing	Retrieve self-owned AMIs and return ID, name, state, architecture, root device and creation date.
FR-03	EC2 Listing	Retrieve EC2 instances and return ID, Name, state, type, IP addresses, AMI and launch time.
FR-04	Key Pair Listing	Retrieve available EC2 key pairs for launch configuration.
FR-05	Security Group Listing	Retrieve security groups and their VPC association.
FR-06	Subnet Listing	Retrieve subnets, availability zones, CIDRs and VPC information.
FR-07	Launch	Accept launch parameters and invoke RunInstancesCommand.
FR-08	Validation	Validate AMI, key pair, subnet, security group and VPC compatibility.
FR-09	DryRun	Perform an AWS DryRun before the actual launch.
FR-10	Tagging	Apply Name and Project=CloudForge tags after successful creation.

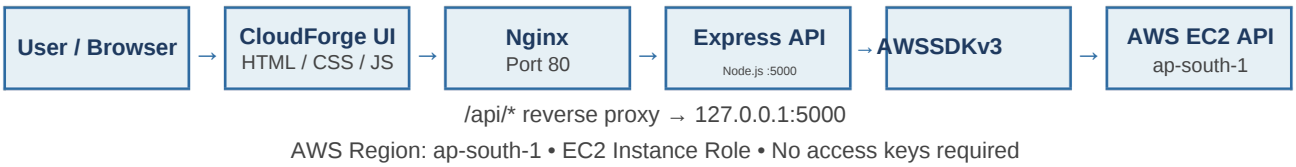
9. Non-Functional Requirements

- **Reliability:** API failures should be surfaced as controlled JSON responses.
- **Security:** AWS credentials should not be hard-coded; use the EC2 IAM role.
- **Maintainability:** frontend, backend, proxy and deployment concerns remain separated.
- **Performance:** API endpoints should query AWS only when required and return concise JSON.
- **Usability:** launch configuration should be presented as a simple form.
- **Availability:** PM2 keeps the Node.js backend running after deployment/restarts.

10. Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Dashboard and user interaction
Backend	Node.js, Express	REST API and business logic
AWS Integration	AWS SDK for JavaScript v3	AWS EC2 API communication
Cloud	Amazon EC2, AMI, VPC	Compute and infrastructure
Identity	IAM EC2 Role	Credential-free AWS API access
Web Server	Nginx	Static hosting and reverse proxy
Process Manager	PM2	Backend process supervision
OS	Ubuntu 24.04	CloudForge host operating system
Region	ap-south-1	AWS Mumbai region
Source Control	Git/GitHub	Versioning and collaboration

11. System Architecture

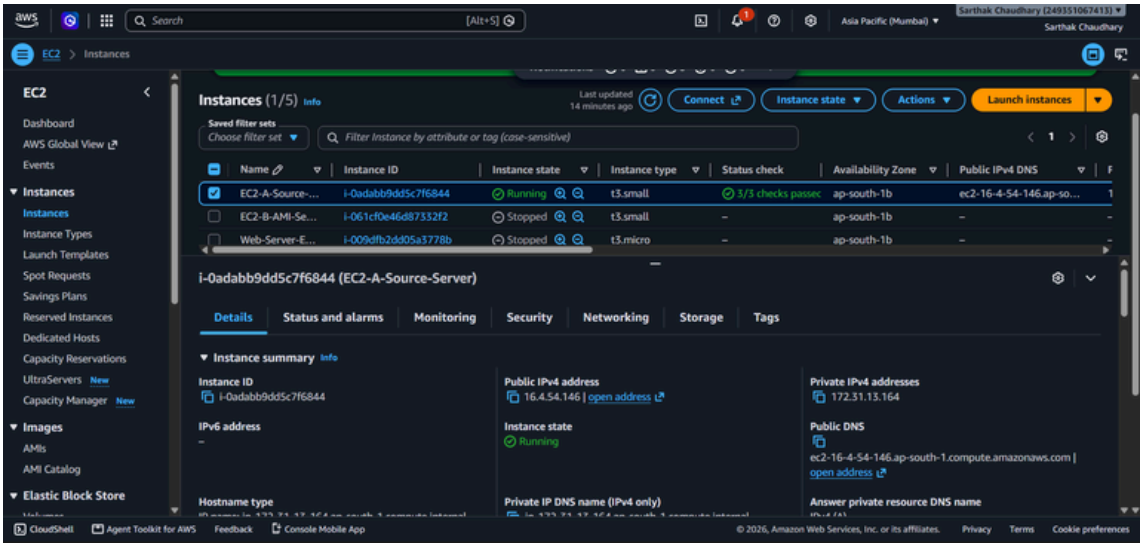


Architecture explanation: The browser loads the CloudForge frontend from Nginx. Requests beginning with /api/ are reverse-proxied by Nginx to the Express service on localhost port 5000. Express uses the AWS SDK v3 and the EC2 instance role to call AWS APIs. Results are converted into compact JSON responses consumed by the frontend.

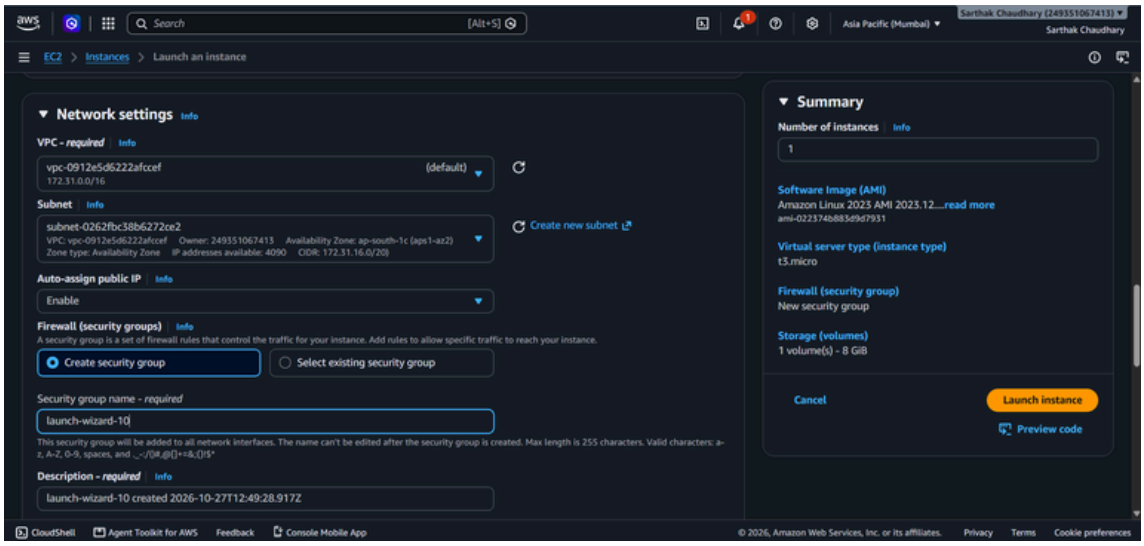
12. AWS Infrastructure and Networking

The working environment uses AWS EC2 as the CloudForge server. The project operates in the Mumbai region (ap-south-1). The EC2 host is attached to the IAM role CloudForge-EC2-Role. The launch workflow uses resources from the same VPC where required.

Verified project resources during implementation: a selected subnet in ap-south-1c with CIDR 172.31.16.0/20 and public-IP mapping enabled; a selected security group named devops-ami-sg; and the key pair Window-machine-demo. The exact IDs are environment-specific and should be treated as deployment values rather than hard-coded application constants.



Screenshot 1: CloudForge EC2 Host Instance

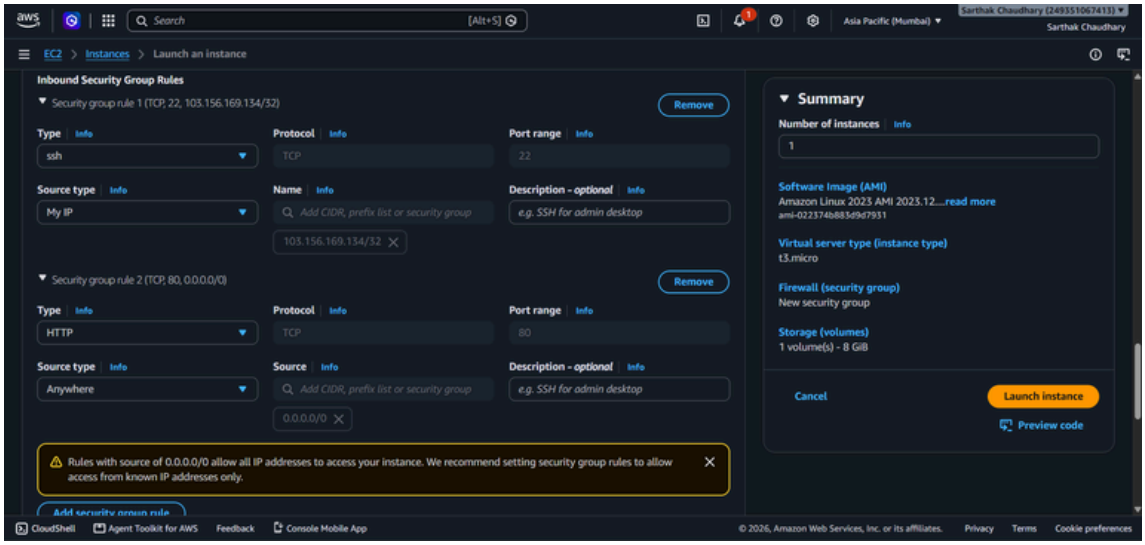


13. IAM and Security Model

The CloudForge server uses an EC2 IAM role rather than storing long-lived AWS access keys on disk. The AWS SDK obtains temporary credentials through the EC2 role mechanism. During development, the role was configured with broad EC2 permissions to establish functionality. For production use, this should be replaced with a least-privilege policy containing only the actions required by the CloudForge endpoints.

Security principle: the frontend never receives AWS credentials. AWS operations occur server-side through the Express API.

Screenshot 3: IAM Role Attached to the CloudForge Host

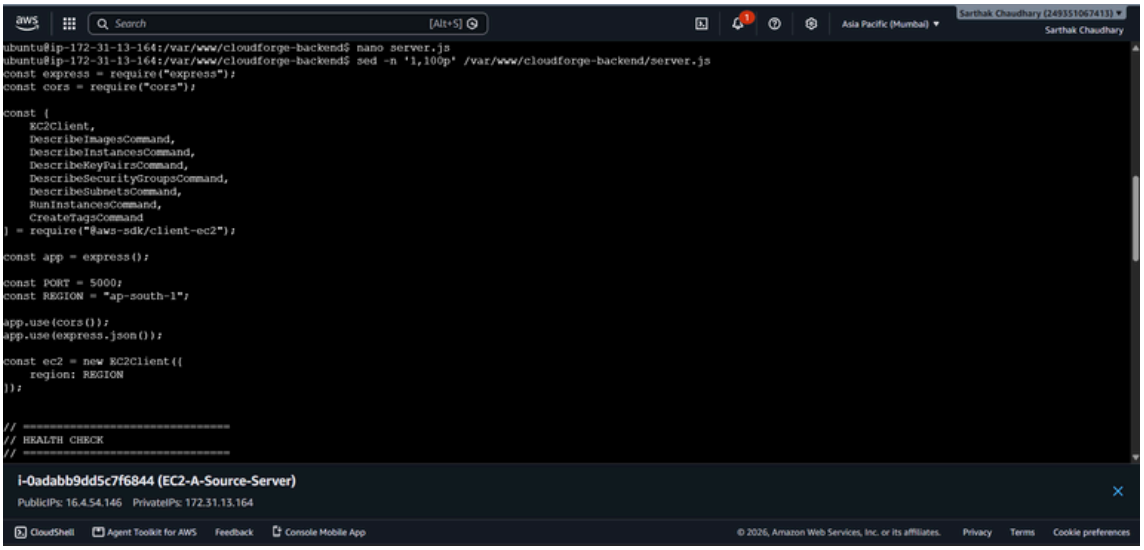


14. Backend Design and API Layer

The backend is located under /var/www/cloudforge-backend and runs on Node.js 18.19.1 in the current implementation. Express handles HTTP requests and express.json() parses JSON request bodies. CORS is enabled for the application.

The EC2 client is initialized with region ap-south-1. The backend imports AWS EC2 commands including DescribeImagesCommand, DescribeInstancesCommand, DescribeKeyPairsCommand, DescribeSecurityGroupsCommand, DescribeSubnetsCommand, RunInstancesCommand and CreateTagsCommand.

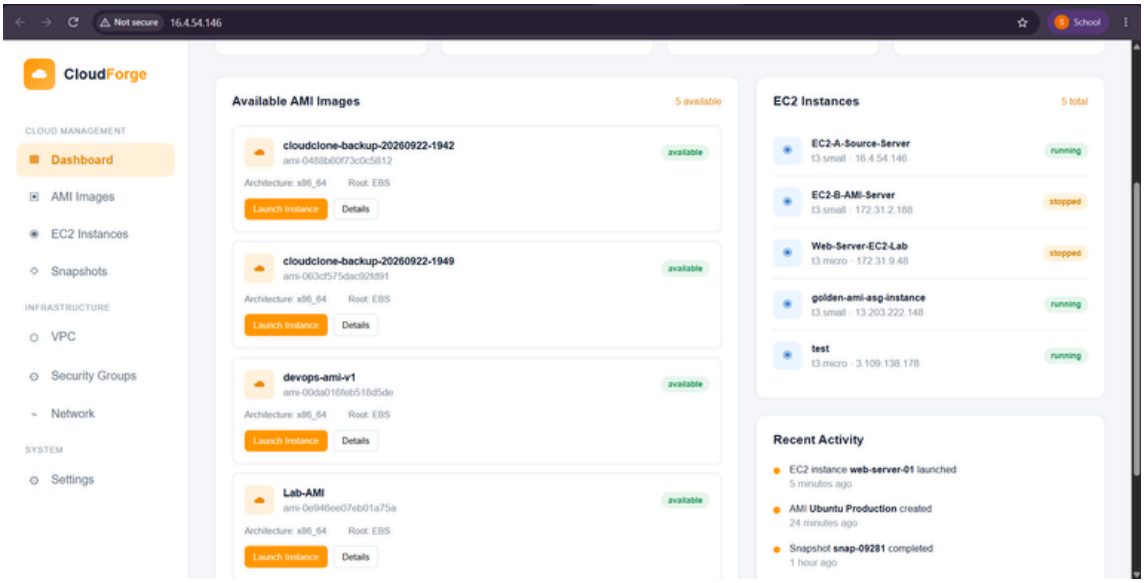
> **Important endpoint behavior:** GET /api/health returns the service status; GET /api/instances retrieves EC2 instances; GET /api/amis retrieves self-owned AMIs; resource endpoints provide key pairs, security groups and subnets; the launch endpoint validates input, performs DryRun, launches the instance and tags it.



15. Frontend Design and Dashboard

The frontend is served from /var/www/aws-ami. Nginx uses this directory as the web root. The interface was designed as a bright, professional dashboard rather than a raw AWS API viewer.

The launch form dynamically loads available AMIs and launch resources. The user selects an AMI, instance type, instance name, key pair, security group and subnet. The browser sends these values to the backend as JSON. The backend remains responsible for validation and AWS execution.

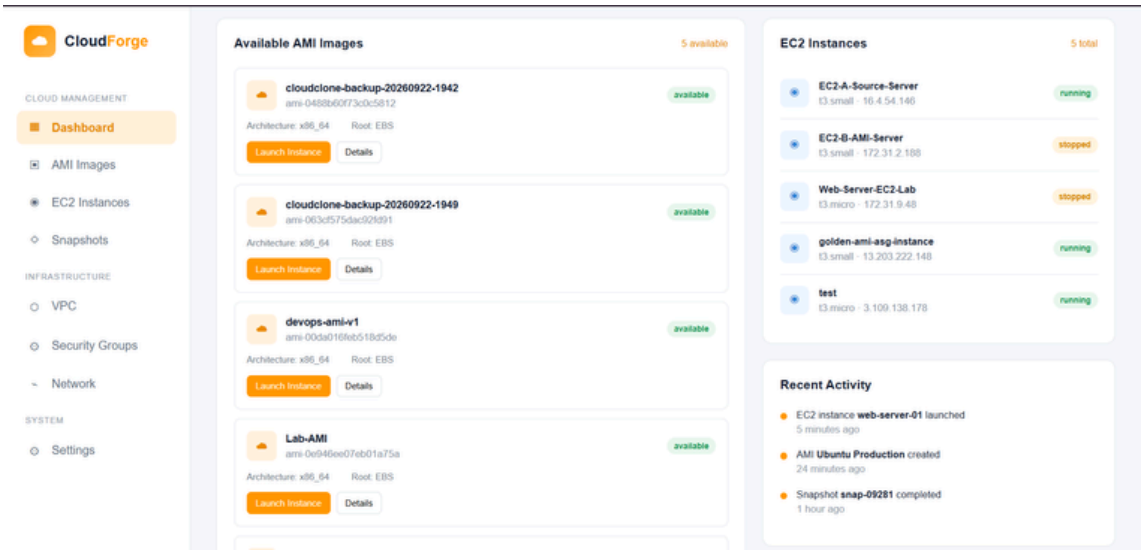


Screenshot 5: CloudForge Dashboard Overview

16. AMI Management Workflow

- User opens CloudForge dashboard.
- Frontend requests GET /api/amis.
- Backend calls DescribeImagesCommand with self-owned AMI filtering.
- AWS returns available AMI metadata.
- Backend maps the AWS response to compact objects containing ID, name, state, architecture, root device and creation date.
- Frontend renders the AMIs in the dashboard.
- When an AMI is created through the application, the user can refresh the list and verify that AWS reports the image as available.

> **Verified example:** the environment returned five available AMIs, including devops-ami-v1, Lab-Instance-Image, two cloudclone-backup images, and Lab-AMI.



GET /api/amis JSON Response



17. EC2 Launch Workflow

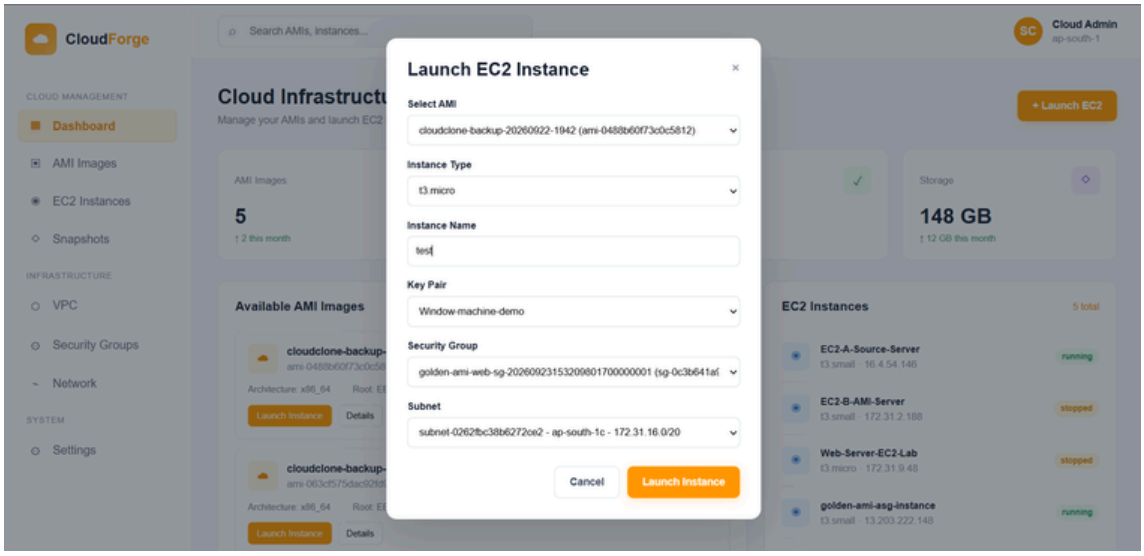


RunInstancesCommand → CreateTagsCommand → Dashboard refresh

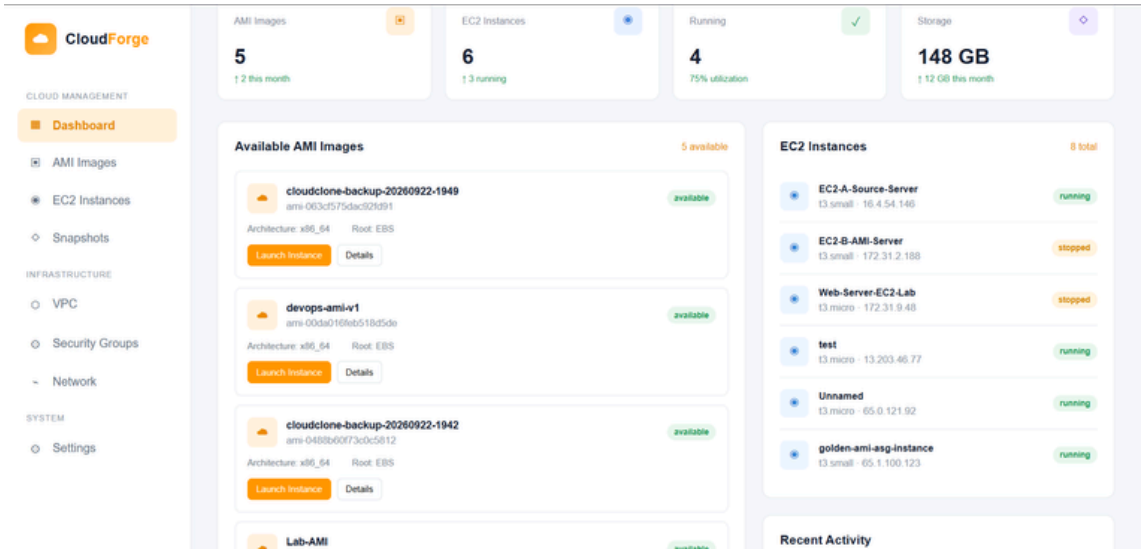
Name = user-provided name (fallback: CloudForge-Instance) | Project = CloudForge

- Select an available AMI.
- Choose the instance type, for example t3.micro.
- Provide an instance name.
- Choose an available key pair.
- Choose a security group.
- Choose a subnet.
- Frontend submits amiId, instanceType, name, keyName, securityGroupId and subnetId.
- Backend validates each selected resource.

- Backend checks subnet/security-group VPC compatibility.
- Backend executes AWS DryRun.
- If DryRun returns DryRunOperation, the request is authorized/configured sufficiently for the next step.
- Backend executes the real RunInstancesCommand.
- If DryRun returns DryRunOperation, the request would be accepted and the workflow proceeds.
- Backend returns the new instance ID and state.
- Frontend refreshes the instance list.



Instance List After Launch



18. Validation and DryRun Mechanism

Validation was added to prevent common configuration errors before a real instance is created. The backend checks whether the AMI exists and is available, verifies the key pair when supplied, checks the subnet, checks the security group, and ensures that a supplied subnet and security group belong to the same VPC. The backend then creates a RunInstancesCommand with DryRun enabled. AWS uses DryRun to test whether the request would be accepted. The application treats the expected DryRunOperation response as a successful preflight result; other errors are returned to the API error path.

This is particularly important because an actual RunInstances request can create billable cloud resources. DryRun provides an additional configuration and authorization checkpoint. A successful DryRun indicates only that the request would be accepted under the tested configuration and authorization conditions; it does not guarantee that the subsequent real launch will succeed, because service quotas (for example, the account's vCPU limit) and other runtime constraints can still cause RunInstancesCommand to fail.

19. Deployment Architecture



Inside the CloudForge server: Nginx · port 80 | PM2 · cloudforge-api

Nginx reverse proxy: /api/* → 127.0.0.1:5000

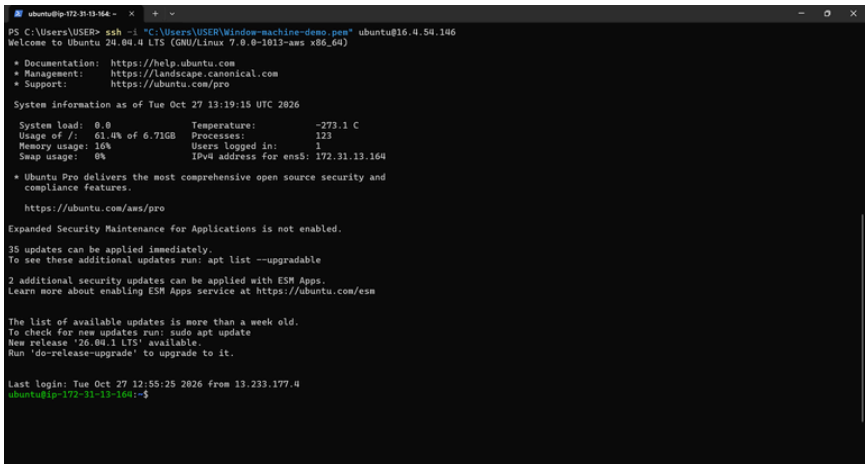
The deployed request path is: Browser → Nginx on port 80 → /api/* reverse proxy → Express on 127.0.0.1:5000 → AWS SDK → AWS EC2 APIs. Static frontend requests are served directly by Nginx. PM2 keeps the Node.js API process running independently of the Nginx service.

20. Complete Step-by-Step Implementation

Step 1 — Create / prepare the AWS EC2 host

Provision an Ubuntu EC2 instance to act as the CloudForge application server. Confirm network access and connect through SSH. The project host used in development was an Ubuntu 24.04 EC2 instance.

SSH Session on the Ubuntu CloudForge Host



Step 2 — Install AWS CLI and verify AWS identity

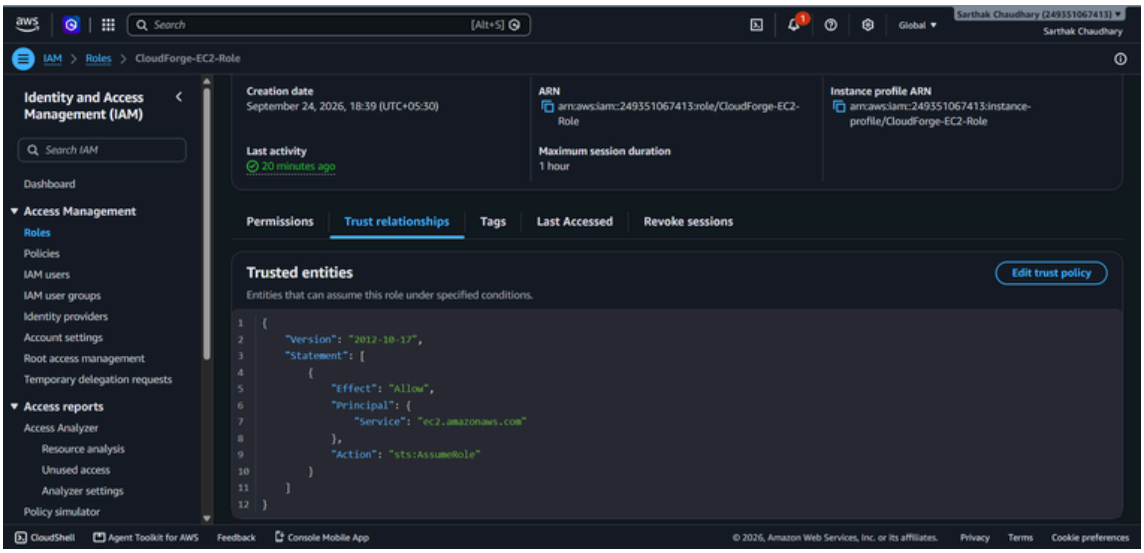
Install AWS CLI v2 and run aws sts get-caller-identity. The command should return the assumed-role identity of the EC2 instance rather than a long-lived IAM access key.

AWS CLI Identity Check

```
ubuntu@ip-172-31-13-164:~$ aws --version
aws-cli/2.37.1 Python/3.14.6 Linux/7.0.0-1013-aws exe/x86_64.ubuntu.24
ubuntu@ip-172-31-13-164:~$ aws sts get-caller-identity
{
    "UserId": "AROATUDT3K4KQC4N3KLDQ:i-0adabb9dd5c7f6844",
    "Account": "249351067413",
    "Arn": "arn:aws:sts::249351067413:assumed-role/CloudForge-EC2-Role/i-0adabb9dd5c7f6844"
}
```

Step 3 — Configure IAM role

Create an EC2-trusted IAM role and attach it to the CloudForge host. The development environment used CloudForge-EC2-Role. Start broad enough to establish the project, then reduce permissions to least privilege before production.



Step 4 — Install Node.js and create backend

Set up the Node.js environment, create /var/www/cloudforge-backend, initialize npm, and install express, cors and @aws-sdk/client-ec2. Create server.js and initialize the EC2Client with the Mumbai region.

```
ubuntu@ip-172-31-13-164:~$ node --version
v18.19.1
ubuntu@ip-172-31-13-164:~$ cd /var/www/cloudforge-backend
ls -la
total 96
drwxr-xr-x  3 ubuntu ubuntu  4096 Oct 27 12:56 .
drwxr-xr-x  5 root   root    4096 Sep 24 13:17 ..
drwxrwxr-x 74 ubuntu ubuntu  4096 Sep 24 13:21 node_modules
-rw-rw-r--  1 ubuntu ubuntu 44602 Sep 24 13:21 package-lock.json
-rw-rw-r--  1 ubuntu ubuntu   343 Sep 24 13:21 package.json
-rw-rw-r--  1 ubuntu ubuntu 11756 Sep 24 15:36 server.js
-rw-rw-r--  1 ubuntu ubuntu  8897 Sep 24 15:11 server.js.backup
-rw-rw-r--  1 ubuntu ubuntu  8897 Sep 24 15:15 server.js.backup-20260924-151526
ubuntu@ip-172-31-13-164:/var/www/cloudforge-backend$ cat package.json
{
  "name": "cloudforge-backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@aws-sdk/client-ec2": "^3.1139.0",
    "cors": "^2.8.6",
    "express": "^5.2.1"
  }
}
```

Step 5 — Implement health endpoint

Add GET /api/health and verify locally using curl against 127.0.0.1:5000. This separates backend availability problems from AWS API problems.

```
ubuntu@ip-172-31-13-164:/var/www/cloudforge-backend$ curl http://127.0.0.1:5000/api/health
{"success":true,"message":"CloudForge API is running","region":"ap-south-1"}ubuntu@ip-172-31-13-164:/var/www/cloudforge-backend$
```

Step 6 — Implement AWS read endpoints

Implement endpoints for instances, AMIs, key pairs, security groups and subnets. Each endpoint uses the AWS SDK to query the current account and maps the response into frontend-friendly JSON.

```
":"available"}]}ubuntu@ip-172-31-13-164:/var/www/cloucurl -s http://127.0.0.1:5000/api/instances | python3 -m json.tool
{
    "success": true,
    "count": 6,
    "instances": [
        {
            "id": "i-0adabb9dd5c7f6844",
            "name": "EC2-A-Source-Server",
            "state": "running",
            "type": "t3.small",
            "privateIp": "172.31.13.164",
            "publicIp": "16.4.54.146",
            "ami": "ami-006f82a1d5a27da54",
            "launchTime": "2026-10-27T12:26:08.000Z"
        },
        {
            "id": "i-061cf0e46d87332f2",
            "name": "EC2-B-AMI-Server",
            "state": "stopped",
            "type": "t3.small",
            "privateIp": "172.31.2.188",
            "publicIp": "-",
            "ami": "ami-00da016feb518d5de",
            "launchTime": "2026-09-24T12:38:33.000Z"
        },
        {
            "id": "i-009dfb2dd05a3778b",
            "name": "Web-Server-EC2-Lab",
            "state": "stopped",
            "type": "t3.micro",
            "privateIp": "172.31.9.48",
            "publicIp": "-",
            "ami": "ami-0faf2b6e64f313b65",
            "launchTime": "2026-09-18T05:22:19.000Z"
        },
        {
            "id": "i-043791a817381c4bc",
            "name": "test",
            "state": "running",
            "type": "t3.micro",
            "privateIp": "172.31.30.69",
```

Step 7 — Build the frontend

Create the dashboard under /var/www/aws-ami. Add AMI and EC2 sections and a launch form. Replace static launch options with data loaded from the backend.

```
ubuntu@ip-172-31-13-164:/var/www/cloudforge-backend$ cd /var/www/aws-ami
ubuntu@ip-172-31-13-164:/var/www/aws-ami$ ls -lah
total 204K
drwxr-xr-x 2 root root 4.0K Sep 24 15:01 .
drwxr-xr-x 5 root root 4.0K Sep 24 13:17 ..
-rw-r--r-- 1 root root  59K Sep 24 13:38 index-backup.html
-rw-r--r-- 1 root root  36K Sep 24 15:01 index-before-launch-options.html
-rw-r--r-- 1 root root  40K Sep 24 15:02 index.html
-rw-r--r-- 1 root root  59K Sep 24 12:46 index.htmlM-b
ubuntu@ip-172-31-13-164:/var/www/aws-ami$ ls -l /var/www/aws-ami/index.html
-rw-r--r-- 1 root root 40006 Sep 24 15:02 /var/www/aws-ami/index.html
ubuntu@ip-172-31-13-164:/var/www/aws-ami$ grep -R "root /var/www/aws-ami" /etc/nginx/sites-enabled/
/etc/nginx/sites-enabled/cloudforge:    root /var/www/aws-ami;
ubuntu@ip-172-31-13-164:/var/www/aws-ami$
```

Step 8 — Configure Nginx

Create the CloudForge Nginx server block. Serve /var/www/aws-ami as the root and proxy /api/ to http://127.0.0.1:5000. Run nginx -t and reload Nginx.

```
ubuntu@ip-172-31-13-164:/var/www/aws-ami$ cd ..
ubuntu@ip-172-31-13-164:/var/www$ cd ~
ubuntu@ip-172-31-13-164:~$ ls -l /etc/nginx/sites-enabled/
total 0
lrwxrwxrwx 1 root root 37 Sep 24 12:56 cloudforge -> /etc/nginx/sites-available/cloudforge
ubuntu@ip-172-31-13-164:~$ sudo cat /etc/nginx/sites-enabled/cloudforge
server {
    listen 80;
    listen [::]:80;

    server_name _;

    root /var/www/aws-ami;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:5000;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
ubuntu@ip-172-31-13-164:~$
```

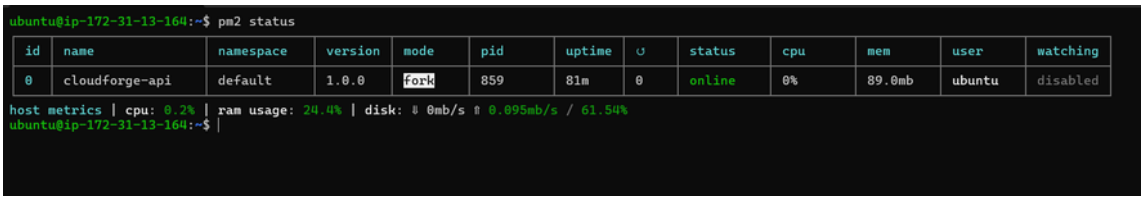
Nginx Configuration Test and Reload

```
ubuntu@ip-172-31-13-164:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-13-164:~$ sudo systemctl reload nginx
ubuntu@ip-172-31-13-164:~$ sudo systemctl is-active nginx
active
ubuntu@ip-172-31-13-164:~$
```

Step 9 — Run the backend with PM2

Start the backend using PM2 with the process name cloudforge-api. Configure PM2 startup so the service can return after a server reboot.

Screenshot 19: PM2 Process Status for cloudforge-api



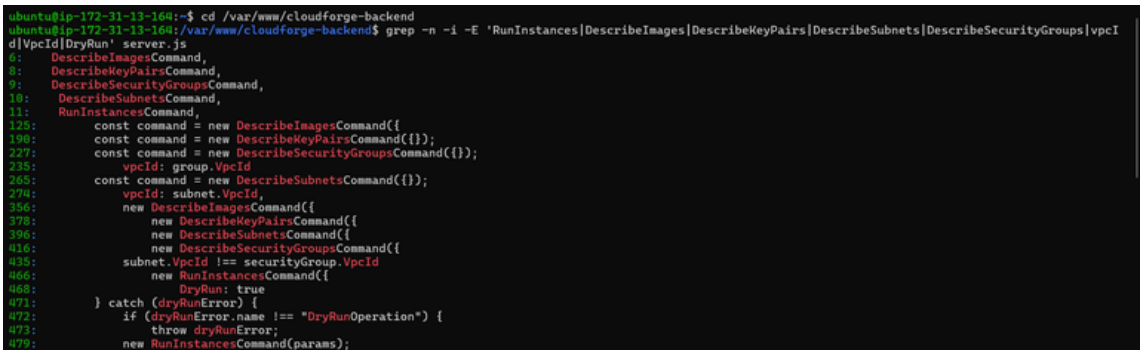
Step 10 — Verify the complete HTTP chain

Test the backend directly and then through Nginx. The important checks are curl http://127.0.0.1:5000/api/health and curl http://localhost/api/health.



Step 11 — Add launch validation

Validate the AMI, key pair, subnet and security group. Check VPC compatibility between subnet and security group.



Step 12 — Add AWS DryRun

Before the real launch, execute RunInstancesCommand with DryRun:true. Continue only when AWS returns the expected DryRunOperation response.

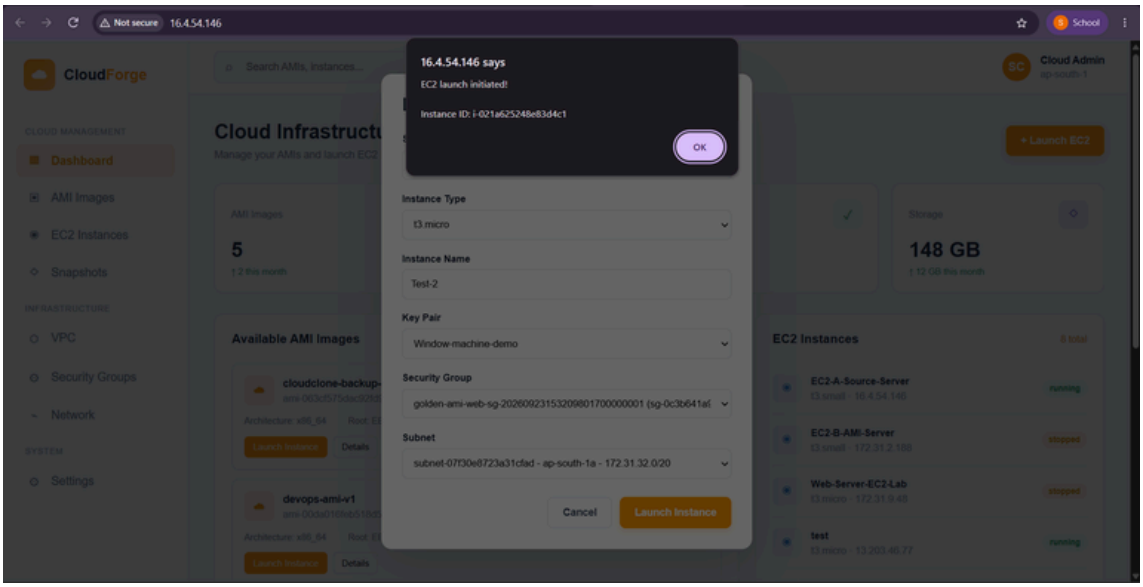
AWS CLI describe-images output (AMI query; no DryRun response shown)



Step 13 — Launch the EC2 instance

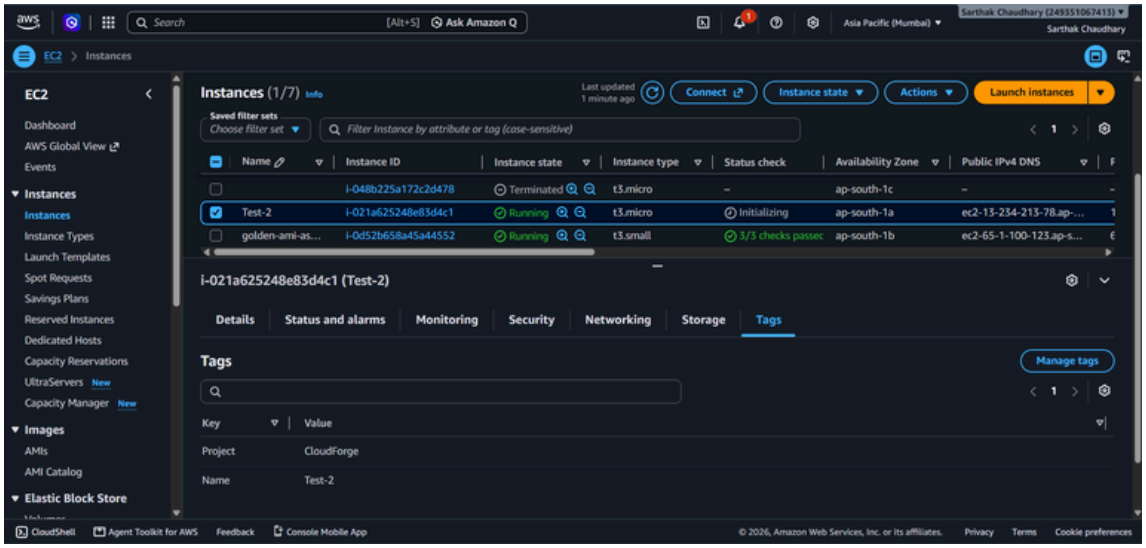
Execute Run Instances Command with Min Count=1 and Max Count=1. Capture the returned Instance Id.

CloudForge dashboard confirming “EC2 launch initiated” for the instance named Test-2



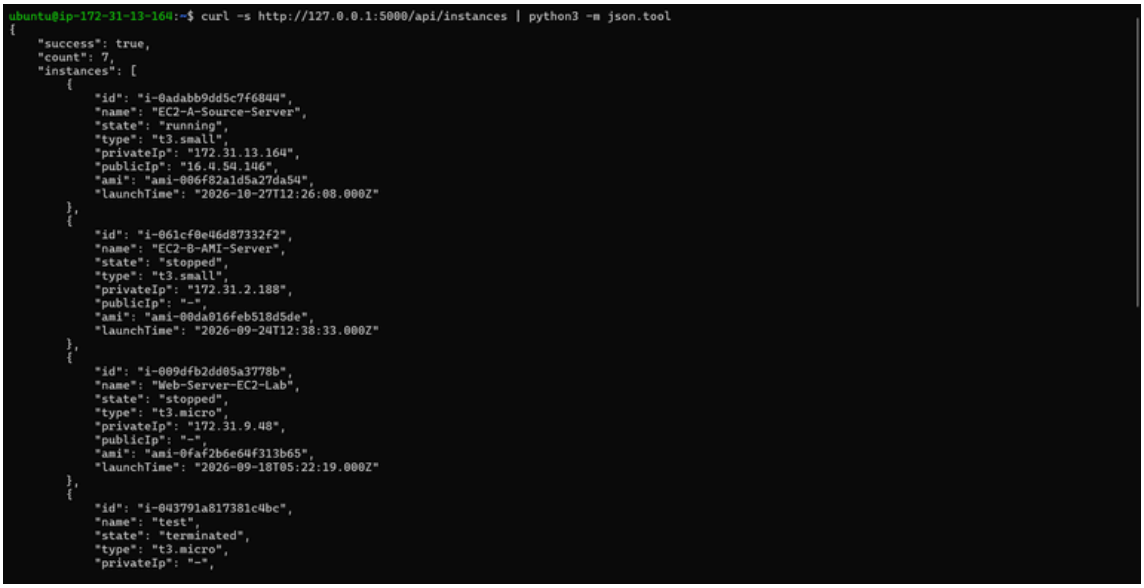
Step 14 — Tag the instance

Execute CreateTagsCommand for the new InstanceId. Apply Name using the submitted name, with CloudForge-Instance as fallback, and apply Project=CloudForge.



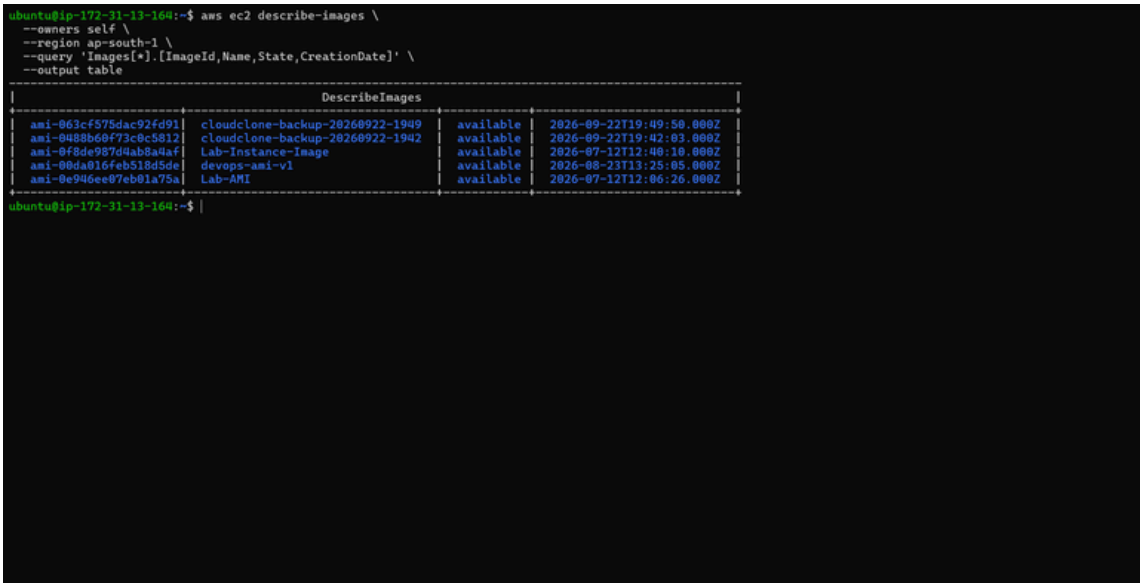
Step 15 — Verify the result

Call GET /api/instances and verify the new instance ID, state, instance type, AMI and Name tag. Confirm the instance in AWS when necessary.



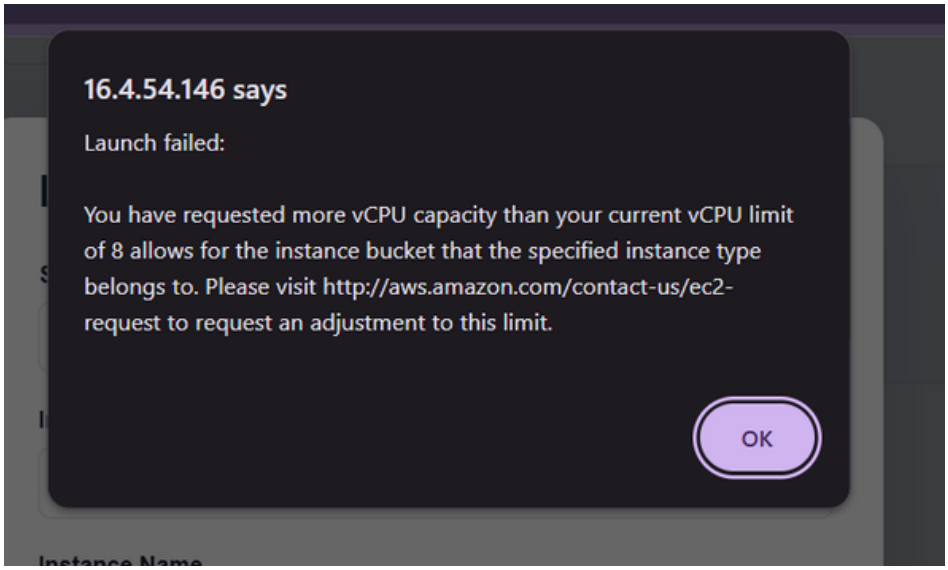
Step 16 — Create and verify AMIs

Use the application’s AMI workflow, then query GET /api/amis to confirm that the newly created image appears with state available.



Step 17 — Test failure paths

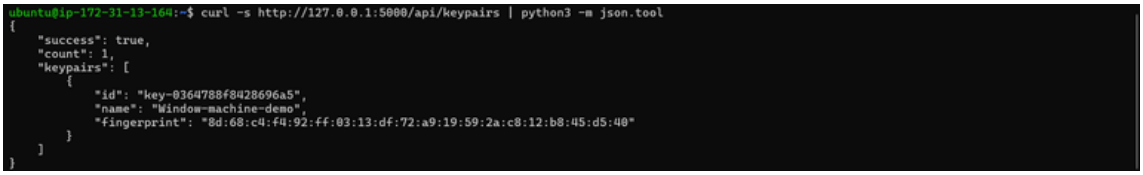
Test invalid AMI IDs, invalid resource IDs, mismatched VPC resources, missing names, unauthorized operations and AWS API errors. Confirm that the frontend receives useful messages.

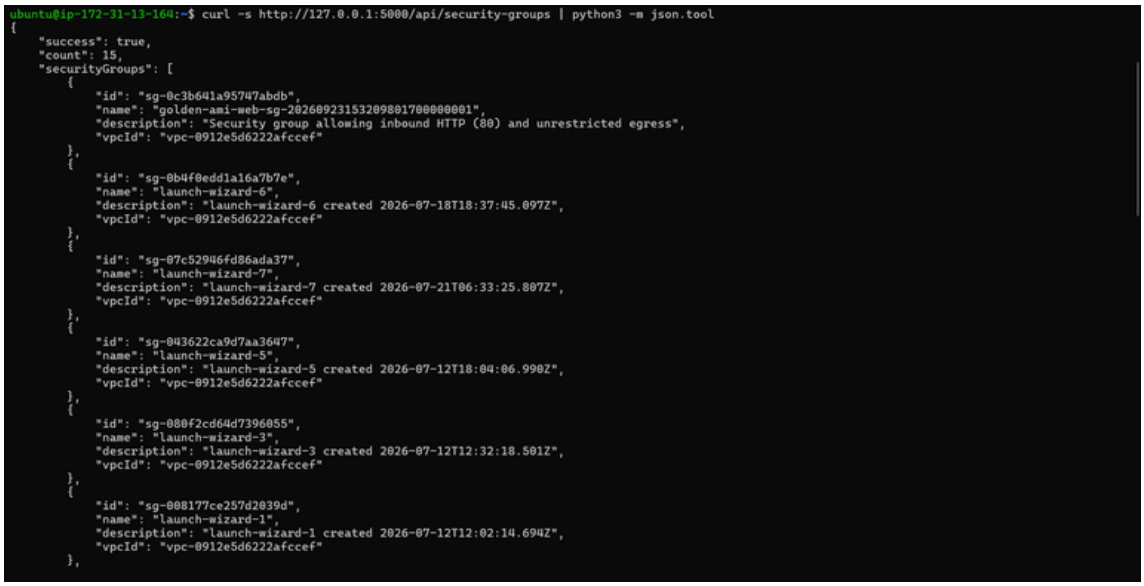


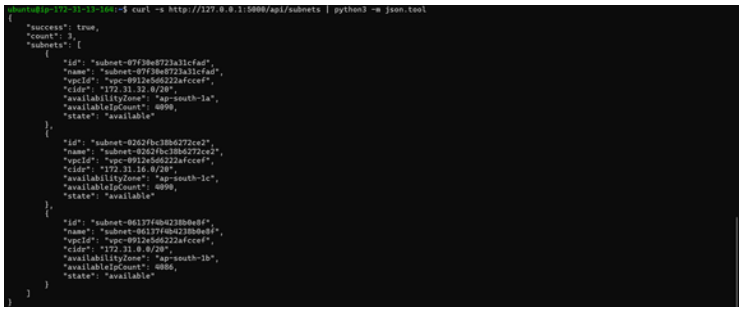
21. Testing and Verification

ID	Test	Method	Expected / Observed	Status
T-01	Backend health	GET /api/health	success=true	**PASS**
T-02	Nginx proxy	GET /api/health through localhost	200 JSON response	**PASS**
T-03	AMI retrieval	GET /api/amis	Five available AMIs observed during verification	**PASS**
T-04	Instance retrieval	GET /api/instances	Current EC2 instances returned	**PASS**
T-05	Key pairs	GET /api/keypairs	Window-machine-demo returned	**PASS**
T-06	Security groups	GET /api/security-groups	Security groups returned	**PASS**
T-07	Subnets	GET /api/subnets	Three subnets returned	**PASS**
T-08	AWS DryRun	run-instances --dry-run	No valid DryRunOperation output captured (CLI attempt used placeholder values)	**NOT VERIFIED**
T-09	Real launch	CloudForge launch	Test-2 launched earlier (Steps 13–14); latest attempt failed: vCPU limit of 8 exceeded	**PASS** latest: BLOCKED
T-10	Manual termination	Terminate test instance	Test instance shown as terminated in AWS console and API output	**PASS**
T-11	Tagging	CloudForge launch after tag fix	Name and Project tags shown on Test-2 (Step 14)	**IMPLEMENTED**

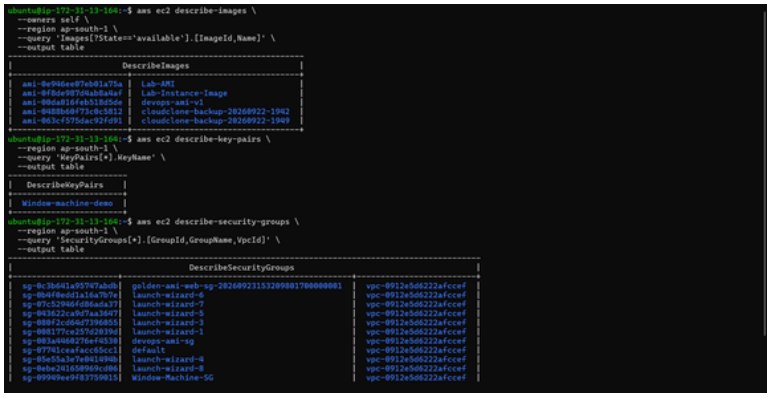
Screenshot evidence for the test cases is placed below. Evidence for T-01 and T-02 is provided by the Step 10 screenshot; for T-03 and T-04 by the AMI workflow and Step 15 screenshots; for T-09 and T-11 by the Step 13 and Step 14 screenshots; and for T-10 by the T-10 screenshot. No valid DryRun output has been captured (T-08). The latest launch attempt was blocked by the AWS vCPU quota (see Section 22).



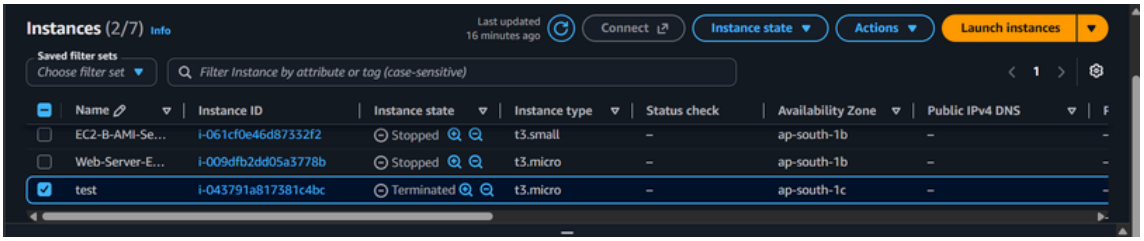




T-08: AWS CLI Resource Query Output (not a DryRun result)



T-10: Test Instance Terminated



22. Problems Encountered and Resolutions

- **API returned frontend HTML:** Nginx initially lacked a /api/ location. Resolution: add an /api/ reverse-proxy block to send requests to port 5000.
- **502 Bad Gateway:** Nginx was correctly proxying but the backend was not running. Resolution: run the Express service under PM2 and verify the local port.
- **Static frontend data:** the dashboard initially contained static content. Resolution: connect frontend functions to live /api/ endpoints.
- **Launch configuration risk:** manually combining AWS resources can create invalid configurations. Resolution: backend validation and subnet/security-group VPC checks.
- **Need for permission/configuration preflight:** real launches create resources. Resolution: AWS DryRun before RunInstancesCommand.
- **Instances displayed as Unnamed:** the launch route did not initially create the Name tag. Resolution: add CreateTagsCommand after successful instance creation.
- **Credential handling:** avoid placing AWS access keys on the server. Resolution: use the EC2 IAM role and AWS SDK credential provider.
- **AWS vCPU quota:** the latest launch failed because the requested vCPU capacity exceeded the account limit of 8 for the instance bucket. Status: unresolved.

23. Current Project Status

- CloudForge frontend is deployed and served through Nginx.
- Express backend is running under PM2.
- Nginx correctly proxies /api/ requests to the backend.
- AWS SDK v3 is connected to EC2 APIs.
- AMI listing is working with real AWS data.
- EC2 instance listing is working with real AWS data.
- Key pairs, security groups and subnets are loaded dynamically.
- Launch validation and AWS DryRun are implemented; a successful DryRunOperation result has not yet been captured as evidence.
- A real EC2 launch was verified earlier (instance Test-2); the latest attempt is blocked by the AWS vCPU quota (limit of 8; unresolved).
- CloudForge Name and Project tags are implemented and were observed on Test-2.

The project is at a functional prototype stage; launching is currently limited by the AWS vCPU quota.

24. Limitations

- The current deployment is a project/demo environment rather than a hardened multi-user production service. EC2 launches are also constrained by the account's vCPU quota (limit of 8).
- The development IAM role was initially broad; least-privilege permissions should be implemented before production.
- The current backend uses Node.js 18; the AWS SDK warning indicates a future upgrade to Node.js 22+ should be planned before the stated support deadline.
- HTTP is currently used; HTTPS should be added for production.
- Authentication and role-based access control for multiple CloudForge users are not yet a complete part of the described implementation.
- Lifecycle operations such as start, stop, reboot and terminate are not yet included in the verified feature set.

25. Future Enhancements

- Add Start, Stop, Reboot and Terminate controls with confirmation dialogs.
- Add instance detail pages including launch configuration and network information.
- Add AMI delete/deregister workflow with safety confirmation.
- Add CloudWatch CPU, network and status monitoring.
- Add storage/EBS information and volume management.
- Introduce user authentication and RBAC.
- Create an audit log of CloudForge actions.
- Implement least-privilege IAM policy.
- Upgrade Node.js runtime to a currently supported LTS version.
- Enable HTTPS with a domain and TLS certificate.
- Add pagination, search and filters for large AWS environments.
- Add multi-region selection.
- Add automated tests and CI/CD deployment.
- Containerize the backend or introduce infrastructure-as-code for repeatable deployment.

26. Project Schedule

Phase	Work	Deliverable
1	AWS setup, EC2, IAM, CLI	CloudForge host + AWS identity
2	Backend foundation	Express API + health check
3	AWS read APIs	AMI/instance/resource endpoints
4	Frontend dashboard	Dynamic CloudForge UI
5	Nginx + PM2	Deployed web application
6	Launch workflow	Validation + DryRun + RunInstances
7	Tagging + verification	Named CloudForge instances
8	Testing	Functional test matrix
9	Hardening	Least privilege + HTTPS + runtime upgrade
10	Future modules	Lifecycle + monitoring + RBAC

27. Conclusion

CloudForge demonstrates an end-to-end cloud management application in which a browser-based interface communicates with a Node.js/Express backend, which in turn communicates securely with AWS EC2 APIs using an IAM role. The implementation covers the complete path from infrastructure setup and API development to deployment, validation, AMI handling and real EC2 provisioning. A later launch attempt was blocked by the account's AWS vCPU quota, which remains unresolved.

The most important engineering characteristic of the project is that the application is not a static mock-up: the dashboard reads live AWS data and the launch workflow performs actual AWS operations. Validation and DryRun reduce configuration risk, while automated tagging improves resource identification. With IAM hardening, HTTPS, lifecycle controls, monitoring, authentication and automated testing, the platform can be extended from a college project into a more complete cloud operations tool.

28. References

- AWS EC2 documentation and AWS SDK for JavaScript v3 EC2 examples.
- AWS IAM documentation for EC2 roles and temporary credentials.
- AWS VPC documentation for subnets, security groups and VPC relationships.
- Node.js and Express documentation.
- Nginx documentation for reverse proxy configuration.
- PM2 documentation for Node.js process management.